

Examen UA1 · Actividad 2: Hilos con IA y skill

Fecha del examen - 04/11/2026

—

Rafael García Adamuz

Programación de servicios y procesos
2º DAM
2026-2027

Esta es mi solución de la Actividad 2 del examen de la UA1. Está formada por una skill sobre procesos e hilos en Java, un programa que simula la venta de entradas de un teatro con cinco taquillas concurrentes y un proceso Auditor independiente, y un informe sobre cómo he usado la IA. Todo el código compila con Java 17, sin librerías externas y en el paquete por defecto.

1. Requisitos y cómo los cumplo

Estos son los requisitos obligatorios del enunciado y dónde se cumple cada uno:

Requisito	Cómo lo cumplo
Cinco taquillas a la vez	SistemaVentas crea cinco hilos Taquilla, los arranca todos con start() en un bucle y solo después los espera con join() en otro.
Cupos 20, 30, 40, 50 y 60	Constante CUPOS en SistemaVentas; cada Taquilla recibe el suyo y vende solo ese número.
Cada venta tarda 5 ms	Thread.sleep(MILIS_POR_VENTA) con la constante a 5 dentro de Taquilla.run().
Sin variables compartidas	Cada Taquilla tiene su cupo y su contador vendidas privados. No hay ningún campo estático modificable.
Leer el resultado al terminar	getVendidas() solo se llama después del bucle de join(), que garantiza que el valor ya es visible.
Salida exacta	Una línea por taquilla, total, tiempo, la línea del Auditor tal cual y Codigo del auditor: C, en ese orden.
Tiempo de ventas	Se mide con System.nanoTime() justo antes del primer start() y justo después del último join().
Proceso independiente	ProcessBuilder lanza otra JVM con la clase Auditor, que tiene su propio main.
Comunicación y código de salida	Se escribe el aforo y el total por su entrada estándar, se lee su respuesta y se obtiene el código con waitFor().
Códigos 0, 1, 2 y 3	El Auditor termina con System.exit y constantes con nombre para cada código.

2. La skill hilos-procesos-java

La skill es el fichero SKILL.md, que está en skill/hilos-procesos-java/ y también en la carpeta de skills de mi agente. Tiene una cabecera con name y description, trece reglas verificables y una lista de comprobación final.

> **name:** hilos-procesos-java
>
> **description:** Guía para escribir programas Java 17 (sin librerías ni paquete) que lanzan procesos independientes con ProcessBuilder y reparten trabajo entre hilos Thread de forma correcta y concurrente. Úsala siempre que haya que crear hilos, esperar a que terminen, medir tiempos de

> ejecución concurrente o comunicarse con otro proceso por su entrada y salida estándar y su código de salida.

Reglas. Hay cinco de procesos, seis de hilos y concurrencia y dos de calidad. Cada una indica cómo se comprueba:

Nº	Tipo	Regla	Cómo se comprueba
1	procesos	**Lanzar con ProcessBuilder y argumentos separados.** Cada argumento va como un elemento distinto (new ProcessBuilder(java, "-cp", classpath, "Auditor")), nunca un único String con espacios.	no hay ninguna cadena con el comando entero.
2	procesos	**No fijar la ruta de Java ni el classpath a mano.** Usar System.getProperty("java.home") + "/bin/java" y System.getProperty("java.class.path") para que el hijo se ejecute con la misma JVM y las mismas clases.	no aparecen rutas absolutas en el código.
3	procesos	**Comunicarse por la entrada y salida estándar y cerrar la escritura.** Se escribe en getOutputStream() y se cierra (try-with-resources) para que el hijo reciba el fin de entrada; después se lee getInputStream() hasta null.	el flujo de escritura se cierra antes de leer.
4	procesos	**Unir la salida de error con la estándar** (redirectErrorStream(true)) y leer toda la salida antes de esperar, para que el hijo no se bloquee con un búfer lleno.	la lectura va antes de waitFor().
5	procesos	**Esperar al hijo y leer su código de salida** con waitFor(), que devuelve el código; el hijo termina con System.exit(codigo) y cada código tiene un significado documentado en una constante.	el padre muestra el valor devuelto por waitFor().
6	hilos	**Arrancar con start(), nunca con run().** Llamar a run() ejecuta el código en el hilo actual y no hay concurrencia.	no se llama a run() directamente.
7	hilos	**Lanzar todos los hilos antes de esperar a ninguno.** Primero un bucle con todos los start() y después otro con todos los join(). Si se hace start() + join() dentro del mismo bucle, el trabajo queda en serie.	el tiempo total se parece al del hilo más lento, no a la suma.
8	hilos	**Cada hilo trabaja con sus propios datos.** Los campos modificables son private y pertenecen solo a ese hilo; no hay variables estáticas ni objetos mutables compartidos entre hilos.	ningún campo static modificable.
9	hilos	**Leer el resultado solo después de join().** join() garantiza que los cambios del hilo ya son visibles para quien lo espera, por lo que no hacen falta synchronized ni volatile.	los getters se llaman después del bucle de join().

10	hilos	**Tratar InterruptedException correctamente.** En sleep() y join() se captura la excepción, se restaura la interrupción con Thread.currentThread().interrupt() y se sale del trabajo; no se deja el catch vacío.	ningún catch vacío.
11	hilos	**Medir el tiempo con System.nanoTime()** justo antes del primer start() y justo después del último join().	la medida abarca todos los hilos y solo ellos.
12	calidad	**Salida exacta y código limpio.** Respetar al pie de la letra el formato de salida pedido, sin tildes en los textos que viajan entre procesos para evitar problemas de codificación, sin números mágicos (aforo, cupos, milisegundos y códigos de salida en constantes), sin paquete y sin ficheros .class en la entrega.	comparar la salida con el enunciado línea a línea.
13	calidad	**Validar la entrada del proceso hijo sin lanzar excepciones.** Si faltan líneas (readLine() devuelve null), el texto no es un número o el valor está fuera de rango, el hijo escribe un veredicto y termina con el código de datos no válidos; no se atrapa NullPointerException.	probar el hijo con entrada vacía, una sola línea, abc y valores negativos y comprobar el código.

Lista de comprobación final. La repasé antes de dar el programa por bueno:

☐ Compila con javac --release 17 sin avisos y sin librerías externas.

☐ Las clases están en el paquete por defecto y cada clase con main tiene el nombre pedido.

☐ Todos los hilos se arrancan con start() antes del primer join().

☐ No hay variables modificables compartidas entre hilos y los resultados se leen tras join().

☐ InterruptedException está tratada sin catch vacíos.

☐ El proceso hijo se lanza con ProcessBuilder, se le escribe por la entrada estándar y se cierra el flujo.

☐ Se lee la salida del hijo completa y se muestra su código de salida con waitFor().

☐ Probados todos los códigos de salida del hijo, incluidos entrada vacía, texto y negativos.

☐ Ejecutado al menos 3 veces: el total es el esperado y el tiempo se parece al del hilo más lento.

☐ La salida coincide exactamente con el formato del enunciado.

☐ No se suben .class, ficheros del IDE ni carpetas del agente.

3. El programa

Son tres clases, cada una en su fichero y sin paquete. Taquilla es el hilo que vende, SistemaVentas coordina todo y Auditor es el proceso independiente.

SistemaVentas.java

Crea las taquillas, las arranca todas, espera a que acaben, escribe el resumen y lanza el Auditor. Para eso usa la misma JVM y el mismo classpath que el programa principal,

escribe los dos datos en la entrada del hijo, lee todo lo que responde y recoge su código de salida.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;

public class SistemaVentas {

        private static final int AFORO = 200;
        private static final int[] CUPOS = {20, 30, 40, 50, 60};

        public static void main(String[] args) throws IOException {
    List<Taquilla> taquillas = new ArrayList<>();
        for (int i = 0; i < CUPOS.length; i++) {
      taquillas.add(new Taquilla(i + 1, CUPOS[i]));
        }

        long inicio = System.nanoTime();
        for (Taquilla t : taquillas) {
      t.start();
        }
        try {
        for (Taquilla t : taquillas) {
        t.join();
        }
        } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      System.err.println("Ventas interrumpidas");
      System.exit(1);
        }
        long milis = (System.nanoTime() - inicio) / 1_000_000;

        int total = 0;
        for (int i = 0; i < taquillas.size(); i++) {
        int vendidas = taquillas.get(i).getVendidas();
      total += vendidas;
      System.out.println("Taquilla " + (i + 1) + ": " + vendidas + " entradas");
        }
    System.out.println("Total vendido: " + total + " de " + AFORO);
    System.out.println("Tiempo de ventas: " + milis + " ms");

        int codigo = lanzarAuditor(AFORO, total);
    System.out.println("Codigo del auditor: " + codigo);
        }

        private static int lanzarAuditor(int aforo, int total) throws IOException {
        String java = System.getProperty("java.home") + "/bin/java";
        String classpath = System.getProperty("java.class.path");
    ProcessBuilder pb = new ProcessBuilder(java, "-cp", classpath, "Auditor");
    pb.redirectErrorStream(true);
        Process proceso = pb.start();

        try (PrintWriter salida = new PrintWriter(
        new OutputStreamWriter(proceso.getOutputStream(), StandardCharsets.UTF_8))) {
      salida.println(aforo);
      salida.println(total);
        }

        try (BufferedReader entrada = new BufferedReader(
        new InputStreamReader(proceso.getInputStream(), StandardCharsets.UTF_8))) {
      String linea;
```

```
            while ((linea = entrada.readLine()) != null) {
               System.out.println(linea);
               }
               }

               try {
            return proceso.waitFor();
               } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            proceso.destroy();
            return -1;
               }
               }
}
```

Taquilla.java

Es un hilo con sus propios datos: el cupo, que no cambia, y el contador de ventas, que solo toca él. Si lo interrumpen mientras duerme, restaura la interrupción y termina.

```
public class Taquilla extends Thread {

               private static final long MILIS_POR_VENTA = 5;

               private final int cupo;
               private int vendidas;

               public Taquilla(int numero, int cupo) {
         super("Taquilla-" + numero);
         this.cupo = cupo;
               }

               @Override
               public void run() {
               for (int i = 0; i < cupo; i++) {
               try {
               Thread.sleep(MILIS_POR_VENTA);
               } catch (InterruptedException e) {
               Thread.currentThread().interrupt();
               return;
               }
            vendidas++;
               }
               }

               public int getVendidas() {
               return vendidas;
               }
}
```

Auditor.java

Lee dos líneas, valida los datos y compara el total con el aforo. Si falta una línea, no es un número o el valor no tiene sentido, termina con el código 3 sin lanzar ninguna excepción.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.nio.charset.StandardCharsets;

public class Auditor {

		private static final int COINCIDE = 0;
		private static final int SOBRAN = 1;
		private static final int FALTAN = 2;
		private static final int INVALIDO = 3;

		public static void main(String[] args) {
	System.exit(auditar());
		}

		private static int auditar() {
		int aforo;
		int total;
		try (BufferedReader entrada = new BufferedReader(
		new InputStreamReader(System.in, StandardCharsets.UTF_8))) {
	    String lineaAforo = entrada.readLine();
	    String lineaTotal = entrada.readLine();
		if (lineaAforo == null || lineaTotal == null) {
		return invalido();
		}
	    aforo = Integer.parseInt(lineaAforo.trim());
	    total = Integer.parseInt(lineaTotal.trim());
		} catch (IOException | NumberFormatException e) {
	    return invalido();
		}

		if (aforo <= 0 || total < 0) {
	    return invalido();
		}
		if (total == aforo) {
	    System.out.println("Correcto: el total coincide con el aforo");
	    return COINCIDE;
		}
		if (total > aforo) {
	    System.out.println("Error: se han vendido mas entradas que el aforo");
	    return SOBRAN;
		}
	System.out.println("Error: se han vendido menos entradas que el aforo");
		return FALTAN;
		}

		private static int invalido() {
	System.out.println("Datos no validos");
		return INVALIDO;
		}
}
```

4. Ejecución y pruebas

Compilé con javac --release 17 y ejecuté SistemaVentas tres veces seguidas. El total fue siempre 200 y el tiempo se parece al de la taquilla más lenta (60 ventas de 5 ms son 300 ms), no a la suma de todas (200 ventas serían unos 1000 ms).

Ejecución	Total vendido	Tiempo de ventas	Código del auditor

1	200 de 200	341 ms	0
2	200 de 200	337 ms	0
3	200 de 200	341 ms	0

También probé el Auditor por separado con todos los casos que se me ocurrieron, incluidos los de datos no válidos:

Entrada (aforo / total)	Respuesta	Código
200 / 200	Correcto: el total coincide con el aforo	0
200 / 210	Error: se han vendido mas entradas que el aforo	1
200 / 150	Error: se han vendido menos entradas que el aforo	2
abc / 1	Datos no validos	3
una sola línea	Datos no validos	3
entrada vacía	Datos no validos	3
200 / -5	Datos no validos	3
0 / 0	Datos no validos	3
99999999999 / 1	Datos no validos	3

Qué demuestra cada prueba. El total de 200 y el código 0 confirman que ninguna venta se pierde. El tiempo de unos 340 ms, cercano a los 300 ms de la taquilla de 60 entradas, confirma que las cinco venden a la vez. Los nueve casos del Auditor cubren los cuatro códigos de salida.

5. Criterios de evaluación

Así se reparte cada criterio de la rúbrica en esta entrega:

Criterio	Puntos	Dónde se cubre
Skill: estructura, descripción y reglas correctas	1,25	Apartado 2: cabecera name y description, trece reglas verificables y lista de comprobación.
Uso de la IA con la skill	0,5	Apartado 1 del informe: instrucciones claras que remiten a la skill. Su efecto se ve en el código (reglas 3 y 7).
Código funcional	1,5	Apartados 3 y 4: hilos a la vez, total de 200, salida exacta y proceso Auditor con sus cuatro códigos.
Revisión crítica e informe	0,75	Informe de la página siguiente: fallos detectados, reglas que influyeron y comprobaciones.

6. Informe de uso de la IA

Alumno: Rafael García Adamuz · **Entrega:** UA1/examen/dev_08/EJ2/

Texto que le di a la IA

> Actúa como desarrollador Java y sigue al pie de la letra las reglas y la lista de comprobación de mi skill hilos-procesos-java. Genera en Java 17, sin librerías externas y sin paquete, un teatro de 200 entradas con cinco taquillas de cupos 20, 30, 40, 50 y 60 que venden a la vez; cada venta es un Thread.sleep(5). Las taquillas son hilos con sus propios datos, sin variables compartidas, y el programa principal lee su resultado solo cuando han terminado. La clase principal es SistemaVentas y debe escribir exactamente, en este orden: Taquilla N: V entradas por taquilla, Total vendido: T de 200, Tiempo de ventas: X ms (desde lanzar las taquillas hasta terminar todas), la salida del Auditor tal cual y Codigo del auditor: C. Después lanza el proceso independiente Auditor (otra clase con su main), envíale por su entrada estándar el aforo y el total en dos líneas y muestra su respuesta y su código de salida: 0 coincide, 1 total mayor, 2 total menor, 3 datos no válidos. Dame solo el código, sin comentarios, y al final indica qué regla de la skill cumple cada parte.

Fallos y mejoras que detecté

Tildes rotas entre procesos. El Auditor escribía Datos no válidos y, al leerlo SistemaVentas, salía Datos no válidos, porque el hijo escribe en la codificación de la consola de Windows y el padre lee en UTF-8. La línea ya no era «tal como la escribe». Lo corregí con mensajes sin tildes en lo que viaja entre procesos y lo añadí a la regla 12.

NullPointerException atrapada. Para detectar la entrada vacía, el Auditor capturaba NullPointerException, que es un parche. Lo cambié por una comprobación explícita de null en las dos líneas leídas y añadí la regla 13 de validación.

Falsa alarma al probar. Desde PowerShell todos los casos del Auditor daban 3 por cómo se pasaba la entrada, no por el código. Repetí las pruebas con tuberías de bash y salieron los códigos esperados.

Cómo influyeron las reglas de mi skill

Regla 7 (lanzar todos los hilos antes de esperar a ninguno): SistemaVentas tiene un bucle con los cinco start() y otro con los cinco join(). Con start() y join() en el mismo bucle las ventas irían en serie y tardarían unos 1000 ms en vez de unos 340 ms.

Regla 3 (cerrar el flujo de escritura): el try-with-resources cierra la entrada del Auditor y este recibe el fin de datos; sin cerrarla se quedaría esperando.

Cómo comprobé que venden a la vez y que el total es correcto

Ejecuté SistemaVentas tres veces: Total vendido: 200 de 200 y Codigo del auditor: 0 siempre.

El tiempo fue de 341, 337 y 341 ms. Se parece a la taquilla más lenta (60 ventas × 5 ms = 300 ms más el arranque) y no a la suma de todas (200 × 5 ms = 1000 ms), así que venden a la vez.

Los resultados se leen después de join() y no hay variables compartidas, por lo que no hay carreras y el 200 no depende de la suerte.

Probé el Auditor con 200/200, 200/210, 200/150, abc/1, una línea, vacía, 200/-5, 0/0 y 99999999999/1, y obtuve los códigos 0, 1, 2, 3, 3, 3, 3, 3 y 3.